

INTERNAL RECRUITMENT SYSTEM (IJP)

Spring Boot Microservices & Angular Full-Stack Architecture
Complete A-to-Z Master Documentation, Viva Defense & Interview Preparation Guide

Project Title: Internal Job Posting / Internal Recruitment System (IJP)

Role & Perspective: Senior Software Architect, Technical Educator & Viva Mentor

Primary Source of Truth: Actual Source Code & Implementation Inspection (No Invented Features)

Backend Architecture: Java 17/24, Spring Boot 3.2.5, Spring Cloud Gateway (:8080), Eureka Server (:8761), OpenFeign

Frontend Architecture: Angular 17+ Standalone Components, TypeScript, RxJS, Bootstrap 5, Vanilla CSS3

Database Engine: MySQL 8.0 — Database-per-Service Pattern (`job\_db`, `candidate\_db`, `admin\_db`)

Target Audience: Trainee Software Developers (Aditya Vikram Singh, Khusboo Yadav, Naman Kapoor)

PROJECT TEAM MEMBERS & PRESENTATION ALLOCATION

Team Member Name	Primary Focus & Architecture Modules	Target Time
Aditya Vikram Singh	System Architecture, API Gateway, Eureka Discovery & Core Java	18–20 Mins
Khusboo Yadav	Candidate Microservice, Employee Registration, Email Login & Validation	18–20 Mins
Naman Kapoor	Job Service, Admin Service, Notifications, Interview & Demo Execution	18–20 Mins

Date of Compilation: August 2026

Document Scope: Comprehensive Architectural Breakdown, End-to-End Traces, UML Diagrams, API Specs, 100+ Interview Q&As; & 1-Hour Presentation Plan

1. Executive Summary & Problem Statement

The **Internal Job Posting (IJP) System** (also referenced as the *Internal Recruitment System*) is a full-stack, enterprise-level microservices application designed specifically for managing internal talent mobility within a corporate organization.

1.1 Real-World Problem Solved

In modern enterprises, hiring internal candidates for promotions or lateral team transfers is often handled manually via scattered email threads, physical noticeboards, or spreadsheets. This manual operational flow suffers from major deficiencies:

- **High External Hiring Costs:** Companies spend significant time and money hiring external candidates when qualified internal employees go unnoticed.
- **Lack of Application Transparency:** Employees have no real-time visibility into their application status, interview schedules, or feedback.
- **Identity & Privacy Violations:** Unstructured systems allow employees to view other candidates' notifications or apply without proper domain verification.
- **Monolithic Fragility:** Traditional single-database recruitment apps suffer total outages when a single sub-system (such as reporting or notifications) experiences high load.

1.2 How IJP Differs from Public Job Portals

Feature Dimension	Public Job Portal (LinkedIn, Indeed)	Internal Job Posting (IJP) System
Target Audience	External job seekers & public recruiters	Current verified company employees only
Authentication Rule	Public emails allowed (@gmail, @yahoo)	Strict company email domain (@company.com)
Identity Mapping	Resume-based public profile	Internal Employee ID ('E901', 'EMP707') & DOB
Data Confidentiality	Publicly search-indexed job posts	Confidential internal pay bands & team transfers
System Boundary	Multi-tenant public cloud infrastructure	Internal microservices architecture with isolated DBs

2. System Requirements Specification (SRS)

2.1 Functional Requirements

1. **Employee Registration:** New employees can register with First Name, Last Name, Date of Birth, Employee ID, Company Email ('@company.com'), and matching passwords. Rejects non-company emails, duplicate emails, and duplicate Employee IDs ('POST /api/candidates/register').
2. **Email-Only Employee Authentication:** Employees log in using company email ('email') and password. Employee ID alone is strictly rejected as a login identifier ('POST /api/candidates/login').
3. **HR Admin Authentication:** Dedicated Admin login using company email ('admin@company.com') and password ('POST /api/admin/login').
4. **Job Posting & Management:** Admins can post new internal vacancies ('POST /api/jobs'), fetch open positions ('GET /api/jobs/open'), toggle job status ('PUT /api/jobs/{id}/status'), and delete job postings ('DELETE /api/jobs/{id}').
5. **Candidate Application Submission:** Logged-in employees can apply for open positions ('POST /api/candidates'). System verifies 'jobId' status is 'OPEN' via OpenFeign inter-service communication and prevents duplicate applications for the same job posting.
6. **Applicant Shortlisting & Interview Scheduling:** Admins can update application status ('APPLIED', 'SHORTLISTED', 'INTERVIEW_SCHEDULED', 'SELECTED', 'REJECTED') and schedule ONLINE (Google Meet) or OFFLINE (Meeting Room) interviews with automated notification creation.
7. **Personalized Employee Notifications:** Logged-in employees receive strictly isolated in-app notification alerts regarding application updates and interview schedules ('GET /api/candidates/notifications/employee/{identifier}').
8. **Cascade Employee Deletion:** Admins can delete a specific candidate record ('DELETE /api/candidates/{id}'), performing a transactionally safe cascade cleanup of associated candidate applications, notifications, and interview records.

2.2 Non-Functional Requirements

- **Scalability & Fault Isolation:** Microservices architecture with Database-per-Service prevents failure in one domain from crashing another.
- **Performance:** Sub-second REST API response times (<500ms) routed through API Gateway.
- **Data Integrity & Isolation:** Strict `@UniqueConstraint` enforcement on `(email, jobId)` and `(employeeId, jobId)`.
- **Aesthetics & UX:** LinkedIn-inspired corporate dark blue UI styling (#0A66C2) with clean Bootstrap 5 layouts.

3. Technology Stack & Technical Rationale

Technology	Version / Component	Where Used in Project	Why Chosen & Problem Solved
Java JDK	17 / 24	Backend core across all microservices	Enterprise stability, strong typing, memory efficiency, & multithreading
Spring Boot	3.2.5	job-service, candidate-service, admin-service	Auto-configuration, embedded Tomcat server, opinionated starters
Spring Cloud Gateway	4.1.x	api-gateway module on port 8080	Single endpoint reverse-proxy, path routing, & central CORS handling
Eureka Server	Spring Cloud Netflix	service-discovery module on port 8761	Dynamic service registration & discovery without hardcoded URLs
OpenFeign	Spring Cloud OpenFeign	candidate-service -> job-service calls	Declarative REST client for inter-service HTTP communication
Spring Data JPA	Jakarta Persistence 3.1	All repositories across microservices	Reduces boilerplate CRUD queries via automated JpaRepository interfaces
Hibernate ORM	6.x	Persistence provider across services	Maps Java entities to MySQL tables and manages transaction boundaries
MySQL Server	8.0	`job_db`, `candidate_db`, `admin_db` schemas	Relational integrity, ACID compliance, and Database-per-Service isolation
Angular Framework	17+ (Standalone)	ijp-frontend application on port 4200	SPA dynamic UI, modular components, RxJS Observables, & Router Guards
Apache Maven	3.9.16	Multi-module build configuration	Dependency management, compilation, testing, and executable JAR packaging

4. System Architecture & Diagrammatic Models

The IJP System implements a **Decoupled Microservices Architecture** following the **Database-per-Service Pattern**. Below are the key architectural diagrams.

4.1 High-Level Component & Routing Diagram



4.2 Microservices vs Monolith Comparison

Architectural Dimension	Monolithic Architecture	IJP Microservices Architecture
Codebase Structure	Single unified codebase containing all features	3 autonomous business microservices + Gateway + Eureka
Database Pattern	Single shared database schema	Database-per-Service (`job_db`, `candidate_db`, `admin_db`)
Deployability	Re-deploying one feature requires redeploying all	Candidate Service can be deployed independently
Fault Tolerance	Memory leak in reporting crashes whole app	Admin Service failure does not affect job application submission
Inter-Service Calls	In-memory Java method calls	OpenFeign REST HTTP calls over network

5. Microservice-by-Microservice In-Depth Analysis

5.1 Service Discovery (`service-discovery`, Port 8761)

Main Class: `ServiceDiscoveryApplication` | **Port:** `8761` | **Role:** Eureka Registry

Eureka acts as a dynamic lookup server. Microservices register their network locations upon startup. Microservices send heartbeat renewals every 30 seconds. If heartbeats fail for 90 seconds, Eureka evicts the instance.

5.2 API Gateway (`api-gateway`, Port 8080)

Main Class: `ApiGatewayApplication` | **Port:** `8080` | **Role:** Single Entrypoint Reverse-Proxy

Spring Cloud Gateway routes incoming frontend traffic from port 8080 to backend microservices using logical names (`lb://JOB-SERVICE`, `lb://CANDIDATE-SERVICE`, `lb://ADMIN-SERVICE`). It centralizes CORS configuration and hides internal ports (8081, 8082, 8083) from external inspection.

5.3 Candidate Service (`candidate-service`, Port 8082)

Main Class: `CandidateServiceApplication` | **Port:** `8082` | **Database:** `candidate\_db`

Manages candidate applications, registration, email-only login, status updates, interview schedules, and personalized in-app notifications.

Key Method Highlights in `CandidateService.java`:

- `registerEmployee(Candidate candidate)`: Validates `@company.com` email, checks duplicate email (`findByEmailIgnoreCase`), checks duplicate Employee ID (`findByEmployeeId`), initializes status = `APPLIED` and `jobId = 0`.
- `loginEmployee(String email, String password)`: Queries candidates strictly using `findByEmailIgnoreCase(email)`. Non-email identifiers (e.g. Employee ID alone) are rejected with HTTP 401 Unauthorized.
- `applyForJob(Candidate candidate)`: Calls `JobServiceClient.getJobById(jobId)` via OpenFeign. Verifies job status is `OPEN`. Enforces duplicate-application prevention for the same job.
- `deleteCandidate(Long id)`: Performs transactional cascade deletion removing notifications, interviews, and candidate profile.
- `getNotificationsForEmployee(String identifier)`: Resolves candidate IDs belonging to employee email/empId and returns strictly isolated notifications.

5.4 Job Service (`job-service`, Port 8081)

Main Class: `JobServiceApplication` | **Port:** `8081` | **Database:** `job\_db`

Manages internal job postings (`JobPosting` entity). Supports posting creation (`POST /api/jobs`), fetching open jobs (`GET /api/jobs/open`), status updates (`PUT /api/jobs/{id}/status`), and job deletion (`DELETE /api/jobs/{id}`).

5.5 Admin Service (`admin-service`, Port 8083)

Main Class: `AdminServiceApplication` | **Port:** `8083` | **Database:** `admin\_db`

Handles HR Admin login authentication (`POST /api/admin/login`) and Designation Master Data management (`/api/admin/designations`). Admin Registration is intentionally **NOT implemented**.

6. Database Design & Entity Relationship Specifications

The IJP System implements three isolated MySQL database schemas following the Database-per-Service pattern.

Database Schema	Table Name	Columns, Keys & Constraints	Entity Mapping
job_db	`job_posting`	`id` BIGINT PK AUTO_INC `job_id` VARCHAR(255) NOT NULL `designation` VARCHAR(255) `description` TEXT `location` VARCHAR(255) `skill_set` VARCHAR(255) `experience` VARCHAR(255) `salary_min` DOUBLE `salary_max` DOUBLE `status` VARCHAR(50)	`JobPosting.java` `@Entity` `@Table(name='job_posting')`
candidate_db	`candidate`	`id` BIGINT PK AUTO_INC `first_name` VARCHAR(255) `last_name` VARCHAR(255) `employee_id` VARCHAR(255) `dob` VARCHAR(255) `email` VARCHAR(255) `password` VARCHAR(255) `job_id` BIGINT `status` VARCHAR(255) Constraints: `(email, job_id)`, `(employee_id, job_id)`	`Candidate.java` `@Entity` `@Table(name='candidate')`
candidate_db	`notification`	`id` BIGINT PK AUTO_INC `candidate_id` BIGINT `title` VARCHAR(255) `message` TEXT `type` VARCHAR(255) `is_read` BIT(1) `created_at` VARCHAR(255)	`Notification.java` `@Entity` `@Table(name='notification')`
candidate_db	`interview`	`id` BIGINT PK AUTO_INC `candidate_id` BIGINT `job_id` BIGINT `interview_date` VARCHAR(255) `interview_time` VARCHAR(255) `interviewer` VARCHAR(255) `interview_mode` VARCHAR(50) `location` VARCHAR(255) `meeting_link` VARCHAR(255) `status` VARCHAR(50)	`Interview.java` `@Entity` `@Table(name='interview')`
admin_db	`admin`	`id` BIGINT PK AUTO_INC `email` VARCHAR(255) UNIQUE `password` VARCHAR(255)	`Admin.java` `@Entity` `@Table(name='admin')`
admin_db	`designation`	`id` BIGINT PK AUTO_INC `name` VARCHAR(255) UNIQUE `status` VARCHAR(50)	`Designation.java` `@Entity` `@Table(name='designation')`

7. End-to-End Execution Trace & Layered Code Architecture

Spring Boot enforces a layered code architecture: **Controller** → **Service** → **Repository** → **Entity** → **Database**.

7.1 Spring Boot Annotation Reference Guide

- `@RestController`: Combines `@Controller` and `@ResponseBody`. Converts Java object return types directly into JSON responses.

- `@Service`: Marks business logic classes. Managed as Spring singleton beans in IoC container.
- `@Repository`: Marks Data Access Objects (DAOs) extending `JpaRepository`. Translates SQL exceptions into Spring Data exceptions.
- `@Entity`: Marks Java class as persistent database table. Required `@Id` primary key.
- `@Autowired`: Injects dependency beans automatically from Spring IoC container.
- `@FeignClient(name = 'job-service')`: Declarative REST client interface for microservice-to-microservice calls.

7.2 Detailed End-to-End Trace: Candidate Application Submission

1. **Angular View (`ApplyComponent`)**: Employee selects job and clicks 'Submit Application'. `onSubmit()` executes.
2. **Angular Service (`CandidateService`)**: `applyForJob(candidate)` makes HTTP POST call to Gateway (`http://localhost:8080/api/candidates`).
3. **API Gateway (`ApiGatewayApplication`)**: Gateway evaluates path `/api/candidates/**` and routes request to `CANDIDATE-SERVICE` on port `8082`.
4. **Controller (`CandidateController.java`)**: Method `@PostMapping applyForJob(@RequestBody Candidate candidate)` receives payload.
5. **Service (`CandidateService.java`)**: Executes business logic. Calls `JobServiceClient.getJobById(jobId)` via OpenFeign.
6. **OpenFeign Call**: Queries Eureka (`:8761`), locates `JOB-SERVICE` (`:8081`), executes `GET /api/jobs/{id}`, and fetches job details.
7. **Status Verification**: `CandidateService` checks if `job.getStatus()` is `"OPEN"`. If closed, throws `RuntimeException("Cannot apply: Job is CLOSED!")`.
8. **Repository (`CandidateRepository.java`)**: Executes `candidateRepository.save(candidate)`. Hibernate generates SQL INSERT into `candidate_db.candidate`.
9. **Response**: Returns saved candidate object with HTTP 201 Created to Angular, updating UI view.

8. REST API Master Specification Table

Method	Gateway Endpoint URL	Target Service	Purpose & Access Control
<code>POST</code>	<code>/api/candidates/register`</code>	Candidate Service (:8082)	Registers new employee. Validates @company.com, email/empld uniqueness.
<code>POST</code>	<code>/api/candidates/login`</code>	Candidate Service (:8082)	Authenticates employee using EMAIL + PASSWORD strictly.
<code>POST</code>	<code>/api/candidates`</code>	Candidate Service (:8082)	Submits candidate job application. Validates OPEN status via Feign.
<code>GET</code>	<code>/api/candidates`</code>	Candidate Service (:8082)	Fetches all candidate applications for Admin Dashboard.
<code>DELETE</code>	<code>/api/candidates/{id}`</code>	Candidate Service (:8082)	Deletes candidate record & cascade deletes notifications/interviews.
<code>PUT</code>	<code>/api/candidates/{id}/status`</code>	Candidate Service (:8082)	Updates candidate status (SHORTLISTED, SELECTED, REJECTED).
<code>POST</code>	<code>/api/candidates/interviews`</code>	Candidate Service (:8082)	Schedules ONLINE or OFFLINE interview & generates notification.
<code>GET</code>	<code>/api/candidates/notifications/employee/{identifier}`</code>	Candidate Service (:8082)	Fetches personalized notifications for logged-in employee.
<code>GET</code>	<code>/api/candidates/notifications/employee/{identifier}/unread-count`</code>	Candidate Service (:8082)	Returns unread notification badge count for employee.
<code>PUT</code>	<code>/api/candidates/notifications/{id}/read`</code>	Candidate Service (:8082)	Marks specific notification as read.
<code>GET</code>	<code>/api/jobs`</code>	Job Service (:8081)	Fetches all internal job postings.
<code>GET</code>	<code>/api/jobs/open`</code>	Job Service (:8081)	Fetches OPEN job postings for home page & apply view.
<code>POST</code>	<code>/api/jobs`</code>	Job Service (:8081)	Creates new internal job posting (Admin only).
<code>DELETE</code>	<code>/api/jobs/{id}`</code>	Job Service (:8081)	Deletes specific job posting record by ID.
<code>POST</code>	<code>/api/admin/login`</code>	Admin Service (:8083)	Authenticates HR Admin credentials (`admin@company.com` / `admin123`).
<code>GET</code>	<code>/api/admin/designations`</code>	Admin Service (:8083)	Fetches designation master data list.
<code>POST</code>	<code>/api/admin/designations`</code>	Admin Service (:8083)	Creates new designation master record.

9. Complete Visual UML Diagrams

Below are the standard UML diagrams representing the system design in Mermaid format.

9.1 Use Case Diagram (Mermaid)

```

```mermaid
graph TD
 Employee((Employee Actor))
 Admin((HR Admin Actor))
 subgraph "Internal Recruitment System (IJP)"
 UC1[Register Employee Account]
 UC2[Login with Company Email]
 UC3[Browse Open Job Postings]
 UC4[Apply for Internal Job]
 UC5[View Application Status]
 UC6[View In-App Notifications]
 UC7[Admin Login]
 UC8[Create/Manage Job Postings]
 UC9[Shortlist / Reject Candidates]
 UC10[Schedule Interviews]
 UC11[Manage Designation Master]
 UC12[Delete Candidates / Jobs]
 end
 Employee --> UC1
 Employee --> UC2
 Employee --> UC3
 Employee --> UC4
 Employee --> UC5
 Admin --> UC7
 Admin --> UC8
 Admin --> UC9
 Admin --> UC10
 Admin --> UC11
 Admin --> UC12
```

```

9.2 Sequence Diagram: Job Application Submission (Mermaid)

```

```mermaid
sequenceDiagram
 actor Employee as Employee User participant Angular as Angular (ApplyComponent)
 participant Gateway as API Gateway (:8080) participant CandSvc as Candidate Service (:8082)
 participant Feign as OpenFeign Client participant JobSvc as Job Service (:8081) participant CandDB as candidate_db MySQL
 Employee->>Angular: Fills application form & submits
 Angular->>Gateway: POST /api/candidates (JSON)
 Gateway->>CandSvc: Proxy request to :8082
 CandSvc->>CandSvc: Validate email @company.com & duplicate check
 CandSvc->>Feign: getJobById(jobId)
 Feign->>JobSvc: GET /api/jobs/{id}
 JobSvc-->>Feign: Return JobPosting object (status='OPEN')
 Feign-->>CandSvc: Job details returned
 CandSvc->>CandSvc: Verify job status == 'OPEN'
 CandSvc->>CandDB: candidateRepository.save(candidate)
 CandDB-->>CandSvc: Saved Candidate entity (ID=17)
 CandSvc-->>Gateway: 201 Created (JSON Response)
 Gateway-->>Angular: HTTP 201 Response
 Angular-->>Employee: Show success notification & redirect
```

```

9.3 Entity Relationship Diagram (ERD - Mermaid)

```

```mermaid
erDiagram
 JOB_POSTING {
 bigint id PK
 string job_id
 string designation
 text description
 string location
 string skill_set
 string experience
 double salary_min
 double salary_max
 string status
 }
 CANDIDATE {
 bigint id PK
 string first_name
 string last_name
 string employee_id
 string dob
 string email
 string password
 bigint job_id
 string status
 }
 NOTIFICATION {
 bigint id PK
 bigint candidate_id FK
 string title
 text message
 string type
 boolean is_read
 string created_at
 }
 INTERVIEW {
 bigint id PK
 bigint candidate_id FK
 bigint job_id
 string interview_date
 string interview_time
 string interviewer
 string interview_mode
 string location
 string meeting_link
 string status
 }
 CANDIDATE ||--o{ INTERVIEW : "receives"
 INTERVIEW ||--o{ CANDIDATE : "referenced_by_job_id"
 JOB_POSTING ..|{ CANDIDATE : "scheduled_for"
```

```

10. Team Division & Master Presentation Plan (~60 Mins Total)

To deliver a flawless 1-hour team presentation, technical topics, presentation responsibilities, live demo execution, and interview focus areas are divided equally among the three team members.

| Team Member | Allocated Topics & Architecture Focus | Time | Live Demo Role | Interview Focus |
|---------------------|---|------------|---|--|
| Aditya Vikram Singh | <ul style="list-style-type: none">• System Architecture & Microservice Rationale• API Gateway Routing & Eureka Discovery• Core Java, Maven Build & Project Setup | 18–20 Mins | Architectural overview, Eureka Dashboard & API Gateway request flow demo | Microservices vs Monolith, Eureka registry, API Gateway routing, Core Java |
| Khusboo Yadav | <ul style="list-style-type: none">• Candidate Microservice Architecture• Employee Registration & Email-Only Login• Validation, Data Isolation & Security• candidate_db JPA/Hibernate Mapping | 18–20 Mins | Employee Registration flow, Email-Only Login, & Employee Dashboard demo | Authentication, Validation logic, JPA/Hibernate, SQL constraints, Security |
| Naman Kapoor | <ul style="list-style-type: none">• Job Service & Admin Service Architecture• Job Applications, Status Updates & Deletion• Interview Scheduling & In-App Notifications• Angular Frontend Architecture & RxJS | 18–20 Mins | Admin Login, Job Posting, Shortlisting, Interview Scheduling & Notifications demo | OpenFeign communication, Admin operations, Angular services/RxJS, Testing |

11. Comprehensive Technical Interview & Viva Preparation (100+ Q&A;)

This section provides 100 categorized interview questions and trainee-friendly answers covering the complete project technology stack.

[Q1 (Core Java)] What is the difference between JDK, JRE, and JVM?

Answer: JDK is the development kit containing compiler (javac) and tools. JRE is the runtime environment providing libraries. JVM is the virtual machine executing bytecode.

[Q2 (Core Java)] What is the difference between String, StringBuilder, and StringBuffer?

Answer: String is immutable. StringBuilder is mutable and non-thread-safe (fast). StringBuffer is mutable and thread-safe (synchronized).

[Q3 (Core Java)] How does HashMap work internally in Java?

Answer: HashMap uses an array of Nodes (hash buckets). It computes hashCode(), determines index via (n-1)&hash;, and handles collisions using LinkedList or Red-Black Tree (threshold > 8).

[Q4 (OOP)] What are the 4 main pillars of OOP and how are they used in IJP?

Answer: Encapsulation (private fields in Candidate entity), Inheritance (Extending JpaRepository), Polymorphism (Overloading candidate search methods), Abstraction (Interfaces for services and repositories).

[Q5 (Core Java)] What is an Exception in Java and how is custom exception handling done in Spring Boot?

Answer: Exceptions disrupt normal program flow. In Spring Boot, custom exceptions thrown in services are caught by `@ControllerAdvice` or caught in Controller try-catch blocks returning `ResponseEntity` with HTTP status codes.

[Q6 (Spring Boot)] What is Dependency Injection (DI) and Inversion of Control (IoC)?

Answer: IoC is the architectural principle where the framework manages object lifecycles. DI is the pattern where Spring injects required bean dependencies into classes using `@Autowired` or constructor injection.

[Q7 (Spring Boot)] What is the purpose of `@SpringBootApplication` annotation?

Answer: It is a combination of `@Configuration` (defines Spring beans), `@EnableAutoConfiguration` (auto-configures Spring Boot starters), and `@ComponentScan` (scans package for components).

[Q8 (Spring Boot)] What are Spring Bean Scopes?

Answer: Singleton (default, single instance per IoC container), Prototype (new instance every request), Request (per HTTP request), Session (per HTTP session).

[Q9 (Spring Boot)] What is the role of `pom.xml` in Spring Boot Maven projects?

Answer: pom.xml (Project Object Model) manages project dependencies, plugin configurations, build lifecycles, and Java compiler versions.

[Q10 (Spring Boot)] How does `application.yml` or `application.properties` work?

Answer: It externalizes application configuration (server port, database URL, credentials, Spring Cloud properties) without recompiling Java code.

[Q11 (REST API)] What is a REST API and what HTTP methods are used in IJP?

Answer: Representational State Transfer API. We use `GET` (fetch data), `POST` (create candidate/job), `PUT` (update status/read state), `DELETE` (remove candidate/job).

[Q12 (REST API)] What is the difference between `@RequestParam` and `@PathVariable`?

Answer: `@PathVariable` extracts values from URL path (`/api/candidates/{id}`). `@RequestParam` extracts query parameters (`/api/jobs?status=OPEN`).

[Q13 (REST API)] What is the difference between HTTP status code 400, 401, 404, and 500?

Answer: 400 = Bad Request (validation failure), 401 = Unauthorized (login failure), 404 = Not Found (missing resource), 500 = Internal Server Error (uncaught backend exception).

[Q14 (Microservices)] What is the Database-per-Service pattern and why did we use it?

Answer: Each microservice owns a dedicated database (job_db, candidate_db, admin_db). It ensures loose coupling and independent database schema updates.

[Q15 (Microservices)] How does OpenFeign work in Spring Cloud?

Answer: OpenFeign is a declarative REST client. Developers define an interface with `@FeignClient(name='job-service')`, and Spring dynamically generates the HTTP implementation.

[Q16 (JPA/Hibernate)] What is ORM (Object-Relational Mapping)?

Answer: ORM maps relational database tables to Java object classes (`@Entity`), allowing developers to query databases using Java objects instead of raw SQL.

[Q17 (JPA/Hibernate)] What is `@UniqueConstraint` in JPA?

Answer: Defines composite table uniqueness constraints in JPA (`@Table(uniqueConstraints = {@UniqueConstraint(columnNames = {'email', 'jobId'})})`).

[Q18 (JPA/Hibernate)] What is the difference between `ddl-auto=update` and `ddl-auto=create-drop`?

Answer: `update` modifies table structures dynamically without deleting existing data. `create-drop` drops tables on application shutdown, losing data.

[Q19 (MySQL)] What is a Primary Key and Foreign Key?

Answer: Primary Key uniquely identifies a table row (`@Id @GeneratedValue`). Foreign Key references a Primary Key in another table to enforce relational integrity.

[Q20 (MySQL)] Why is `job_id` stored as a plain `BIGINT` in `candidate` table instead of `@ManyToOne`?

Answer: In microservices, tables reside in separate databases (`job_db` vs `candidate_db`). Foreign Key relationships cannot span different physical databases across services.

[Q21 (Angular)] What is Angular Standalone Components?

Answer: Introduced in Angular 14+, standalone components manage their own imports (`imports: [CommonModule, FormsModule, RouterLink]`) without requiring NgModule declarations.

[Q22 (Angular)] What is an Observable and RxJS in Angular?

Answer: An Observable represents a stream of asynchronous data events over time. Angular HttpClient methods return Observables which components subscribe to using `.subscribe()`.

[Q23 (Angular)] What is the purpose of `AuthService` in Angular?

Answer: Centralizes user authentication state, handles login API calls, stores user session in `localStorage`, and provides `currentUser$` stream to components.

[Q24 (Angular)] What is Angular Router Guard (`canActivate`)?

Answer: Interceptors that control page access based on user session roles (`employeeGuard`, `adminGuard`). If unauthorized, redirects user to `/login`.

[Q25 (Angular)] What is Two-Way Data Binding in Angular?

Answer: Syncs property values between Component TypeScript code and HTML template view automatically using `[(ngModel)]`.

[Q26 (Project)] How is duplicate candidate registration prevented in Candidate Service?

Answer: CandidateService checks `findByEmailIgnoreCase(email)` and `findByEmployeeId(empld)`. If either exists, throws 400 Bad Request `**An employee with this email or employee ID already exists.**`

[Q27 (Project)] Why is Employee ID NOT allowed for Employee Login?

Answer: To enforce strict single-email identity (`@company.com`). Logging in via Employee ID created identity ambiguities across multi-job application candidate rows.

[Q28 (Project)] How are notifications personalized for the logged-in employee?

Answer: Backend endpoint `GET /api/candidates/notifications/employee/{identifier}` resolves all candidate application IDs belonging to the employee email/empld and returns notifications strictly for those IDs.

[Q29 (Project)] What happens when an admin deletes a candidate record?

Answer: `CandidateService.deleteCandidate(id)` executes a `@Transactional` operation that deletes candidate notifications from `notification`, interviews from `interview`, and candidate profile from `candidate` table.

[Q30 (Project)] How does the system prevent applying for a closed job?

Answer: `CandidateService` calls `JobServiceClient.getJobById(jobId)`. If `job.getStatus()` is `'CLOSED'`, it throws a runtime exception and rejects the application.

12. 'Explain Your Project' Pitches & Final Revision Cheat Sheet

12.1 'Tell Me About Your Project' (30-Second Elevator Pitch)

'The Internal Recruitment System (IJP) is a full-stack microservices portal built with Spring Boot, Spring Cloud Gateway, Eureka, MySQL, and Angular 17. It digitalizes internal talent mobility for corporate employees. It enforces strict company email authentication (@company.com), prevents duplicate job applications, manages interview scheduling, and isolates employee notifications across autonomous microservice databases.'

12.2 'Tell Me About Your Project' (2-Minute Summary for Each Presenter)

Aditya's 2-Minute Script (Focus: Architecture & Infrastructure):

'Our project is an Internal Recruitment System built with Spring Boot Microservices and Angular 17. I led the system architecture design. We implemented a microservices pattern with Spring Cloud Gateway running on port 8080 as our endpoint reverse-proxy and Netflix Eureka Server on port 8761 for dynamic service discovery. Each business service—job-service, candidate-service, and admin-service—owns a dedicated MySQL database following the database-per-service pattern. This ensures fault isolation, loose coupling, and independent microservice scalability.'

Khusboo's 2-Minute Script (Focus: Candidate Service & Registration/Auth):

'I focused on the Candidate Microservice, candidate registration, and authentication mechanics. I implemented employee registration with strict company email validation (@company.com) and duplicate protection against existing emails and employee IDs. I also refactored employee login to authenticate strictly via company email and password. Furthermore, I built our personalized notification engine which isolates in-app notifications per employee session without cross-employee data exposure.'

Naman's 2-Minute Script (Focus: Job/Admin Services & Frontend/Demo):

'I worked on the Job Microservice, Admin Microservice, and Angular frontend integration. I implemented job posting CRUD, job status toggling, and safe job deletion. On the admin side, I developed Designation Master data management and candidate interview scheduling (ONLINE Google Meet vs OFFLINE room location). On the frontend, I built our Angular standalone components and RxJS services styled with a professional LinkedIn dark blue theme.'

12.3 'Night Before Interview' 1-Page Revision Cheat Sheet

| Component / Topic | Key Technical Details & Values to Remember |
|---------------------|--|
| Service Discovery | Eureka Server Port: 8761 `@EnableEurekaServer` URL: `http://localhost:8761` |
| API Gateway | Spring Cloud Gateway Port: 8080 Routes: `/api/jobs/**`, `/api/candidates/**`, `/api/admin/**` |
| Job Service | Port: 8081 DB: `job_db` Table: `job_posting` Entity: `JobPosting.java` |
| Candidate Service | Port: 8082 DB: `candidate_db` Tables: `candidate`, `notification`, `interview` |
| Admin Service | Port: 8083 DB: `admin_db` Tables: `admin`, `designation` Admin Login: `admin@company.com` |
| Frontend App | Angular 17+ Port: 4200 Standalone Components Theme Colors: `#0A66C2`, `#0B2A4A`, `#F7F9FC` |
| Employee Login Rule | EMAIL ONLY (`@company.com`) + Password. Employee ID alone is NOT accepted for login. |
| Registration Rules | Requires First Name, Last Name, DOB, EmpID, Email (`@company.com`), Matching Passwords. Rejects duplicate email/empld. |
| Feign Client Call | `CandidateService` calls `JobServiceClient.getJobById(jobId)` -> `JOB-SERVICE` to verify OPEN status. |
| Database Schemas | 3 separate schemas: `job_db`, `candidate_db`, `admin_db` (Database-per-Service pattern). |